



Unit 4

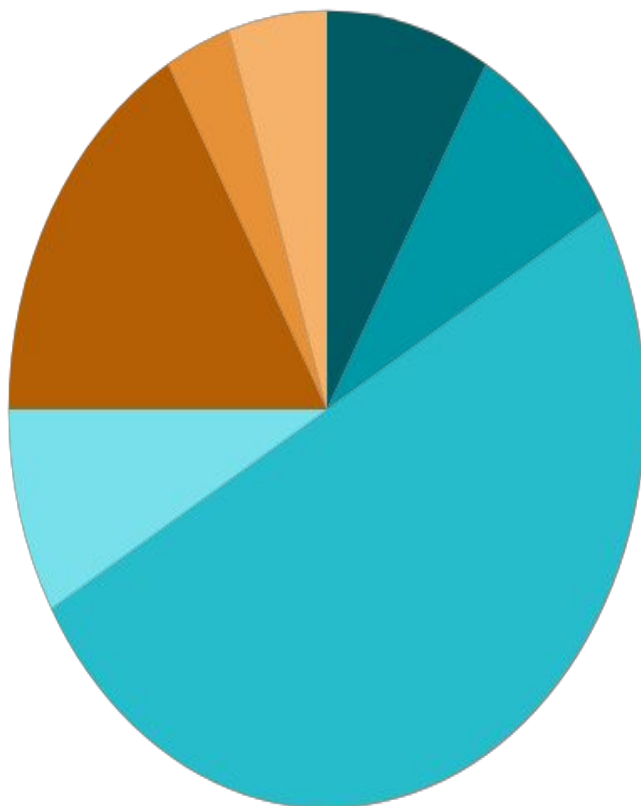
MapReduce



Outline

- Map Reduce workflows
- Unit tests with MR Unit
- Test data and local tests
- Anatomy of Map Reduce job run
- Classic Map-reduce,
- YARN
- failures in classic Map-reduce and YARN
- job scheduling, shuffle and sort, task execution
- Map Reduce types, input formats, output formats

Lecture Structure



	05 Minutes	Outline & Objectives
	05 Minutes	Recap of Previous Lecture
	30 Minutes	Delivery of Actual Content as per Lesson Plan
	05 Minutes	Industry Relevance
	10 Minutes	Interactive Activities
	02 Minutes	Importance of Discipline
	03 Minutes	Recording of Attendance

Recap of Prev Lecture

MapReduce in Hadoop

Overview

- Java-based distributed execution framework within Apache Hadoop.
- Simplifies distributed programming using **Map** and **Reduce** steps.
- Uses **Hadoop Distributed File System (HDFS)** for input & output.

MapReduce Process

- **Map Phase:**
 - Splits data for parallel processing.
 - Applies transformation logic to each chunk.
- **Reduce Phase:**
 - Aggregates the processed data.

Map Reduce workflows

Architecture

- **JobTracker (Master):** Schedules and monitors tasks.
- **TaskTracker (Slaves):** Executes assigned tasks.

Key Characteristics

- Handles **petabytes to exabytes** of data.
- **Write-once, read-many** model.
- Parallel execution without **mutexes**.
- Simple **Map** and **Reduce** operations.
- All **Map** tasks must finish before **Reduce** begins.
- Configurable **number of Map and Reduce tasks**.

Configurable number of Map and Reduce tasks

Task	Formula	Calculation	Final Value
Map Tasks	Total Data Size / Input Split Size	2048MB / 128MB	16 Mappers
Reduce Tasks	Total Data Size / Reducer Input Size	2048MB / 256MB	8 Reducers

Configurable number of Map and Reduce tasks

Parameter	Value
Total Mapper Output Data	2GB (2048MB)
Number of Reducers	8

Using the Formula:

Reducer Input Size = $2048\text{MB} / 8 = 256\text{MB}$

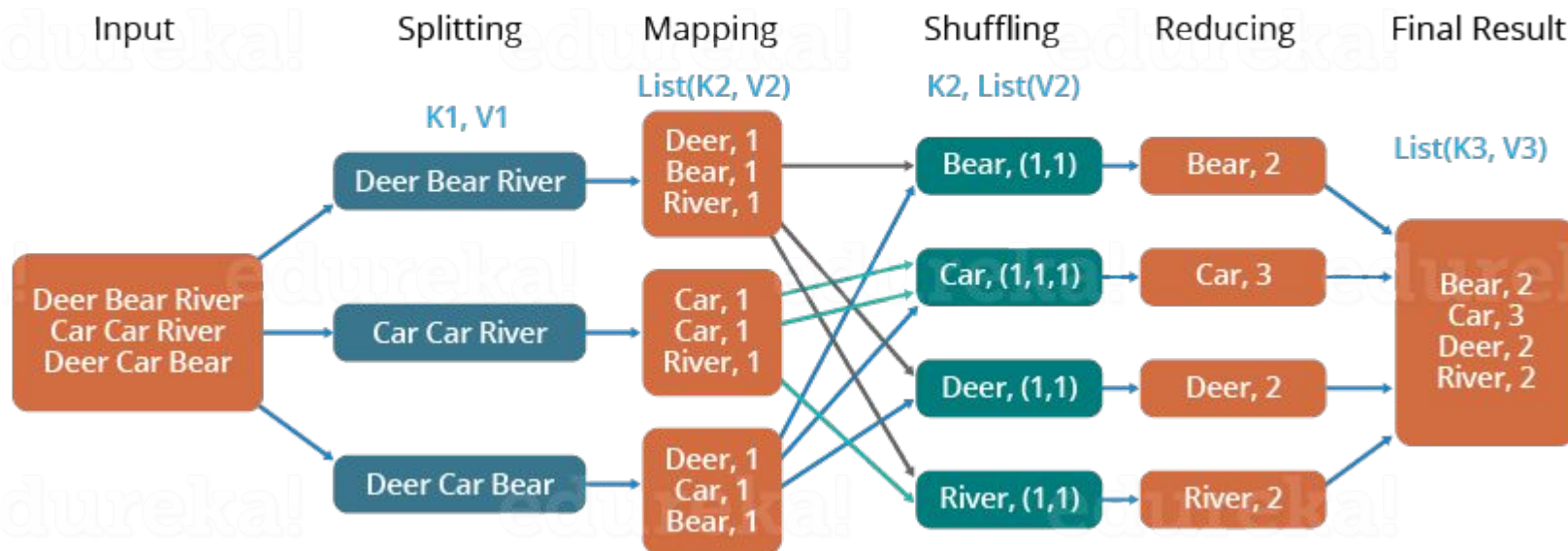
✓ Each Reducer gets 256MB of data to process.



Map Reduce workflows

The Overall MapReduce Word Count Process

edureka!





MapReduce Workflow

- 1 **Input** – Data to be processed
- 2 **Split** – Data is divided into smaller "splits"
- 3 **Map** – Processes each split using the map() function
- 4 **Combine (Optional)** – Reduces data size before shuffling
- 5 **Shuffle & Sort** – Groups and orders data before reducing
- 6 **Reduce** – Aggregates mapper outputs using reduce() function
- 7 **Output** – Final result stored in HDFS



Job Tracker (Master Node)

- Runs on a **single master node**

Functions:

- Accepts jobs from clients & splits them into tasks
- Schedules tasks on worker nodes (Task Trackers)
- Monitors Task Trackers through heartbeat signals
- Reschedules tasks on alternate nodes if a worker fails

Task Tracker (Worker Nodes)

- Runs on **each worker node**

Functions:

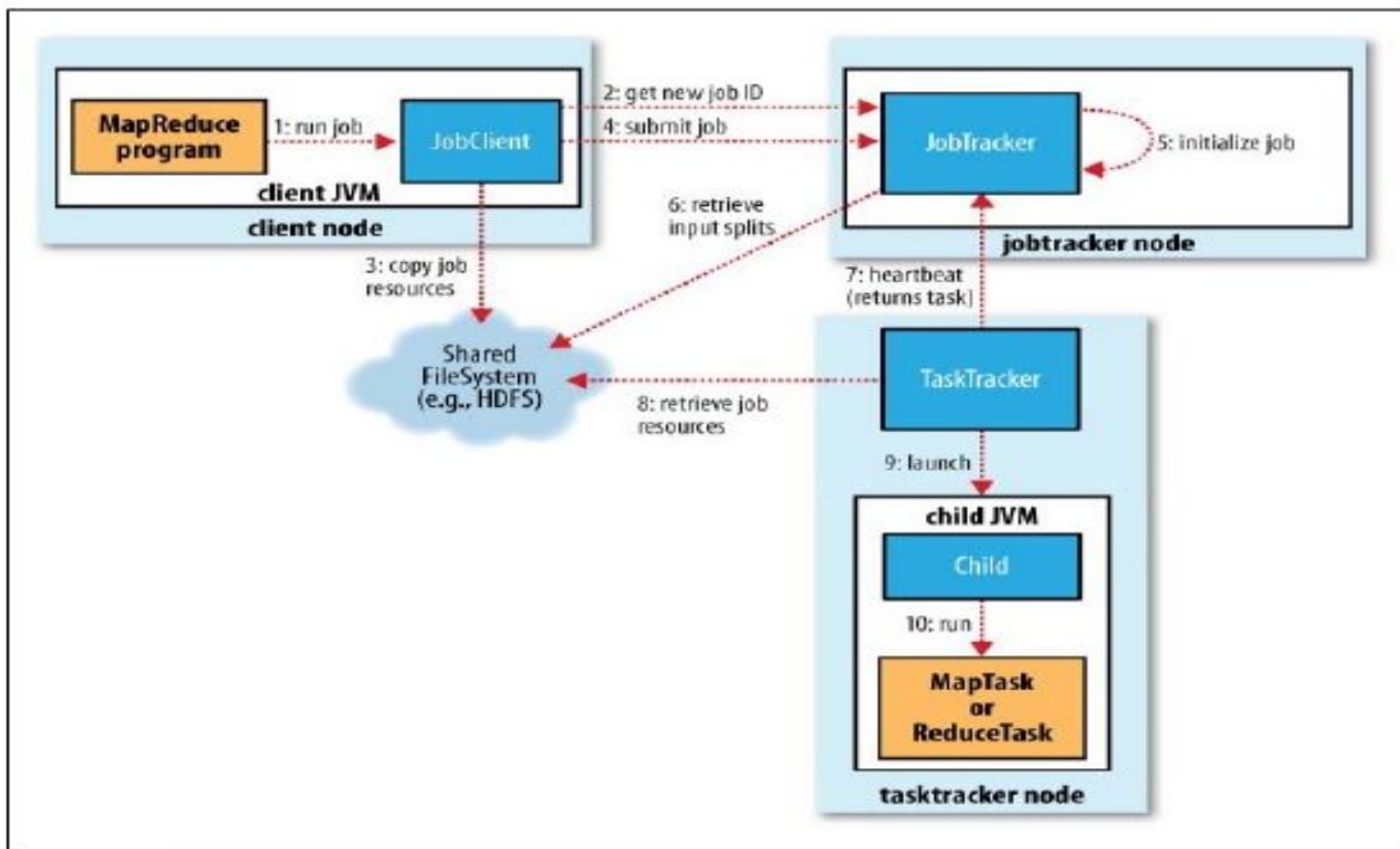
- ◆ Takes assignments from Job Tracker
- ◆ Executes tasks locally in parallel
- ◆ Each worker has a **limited number of mappers & reducers**
- ◆ Prefers more **map jobs** than reduce tasks
- ◆ Runs **task attempts** before final execution
- ◆ Uses **umbilical protocol** to maintain connection with tasks
- ◆ Sends **heartbeat signals** to Job Tracker for status updates
- ◆ Runs tasks in **separate JVMs** to prevent failure propagation



Limitation of MapReduce

- **No Control Over Execution Order**
Cannot determine the order in which Maps or Reduces run
- **Stateless Processing for Maximum Parallelism**
Maps & Reduces must not depend on data from the same job
- **Slower Than Indexed Databases**
A database with an index is faster than MapReduce on raw data
- **Reduce Phase Starts Only After All Maps Finish**
No intermediate reducing, leading to potential delays
- **Assumes Output is Smaller Than Input**
Large datasets generate smaller final results

Anatomy of Map Reduce job run



Anatomy of Map Reduce job run

1. Independent Entities

- **Client:** Submits the MapReduce job.
- **YARN Resource Manager:** Coordinates allocation of compute resources.
- **YARN Node Managers:** Launch and monitor compute containers.
- **MapReduce Application Master:** Manages job execution.
- **Distributed File System:** Stores job files and input/output data.



Anatomy of Map Reduce job run

2. Job Submission

- submit() method creates JobSubmitter instance.
- JobSubmitter performs:
 - Requests new application ID.
 - Validates job output specification.
 - Computes input splits.
 - Copies required resources to shared filesystem.
 - Calls submitApplication() on Resource Manager.
- waitForCompletion() polls progress and reports status.

Anatomy of Map Reduce job run

3. Initialization

- Resource Manager hands request to YARN scheduler.
- Scheduler allocates container and launches Application Master.
- Application Master initializes:
 - Job tracking objects.
 - Retrieves input splits.
 - Creates map and reduce task objects.



Anatomy of Map Reduce job run

4. Task Assignment

- If job does not qualify for Uber mode, Application Master requests containers.
- Map tasks are prioritized before reduce tasks.
- Reduce tasks requested after 5% of map tasks complete.

5. Task Execution

- Application Master requests node managers to launch containers.
- YarnChild process executes the task:
 - Localizes job resources.
 - Runs map or reduce task.
- Streaming mode:
 - Uses standard input/output for communication with external processes.

Anatomy of Map Reduce job run

6. Progress & Status Updates

- Jobs and tasks track and report progress.
- Status includes:
 - Task state.
 - Completion progress.
 - Job counters and status messages.
- Reduce task progress estimated based on phases.

7. Job Completion

- Application Master updates job status to "Successful".
- Job history is archived for future reference.
- OutputCommitter's commitJob() method is called.
- Containers and Application Master clean up resources.

YARN

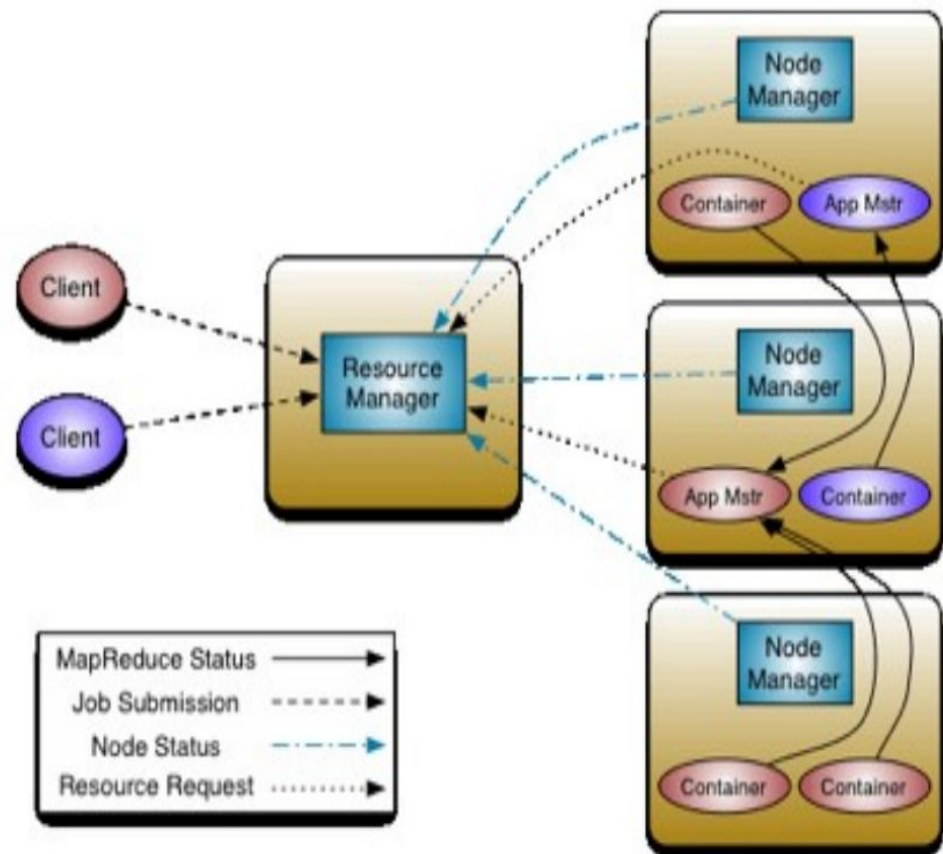
- **Yet Another Resource Negotiator (YARN)** is a large-scale **distributed operating system** for big data applications.
- Supports **various programming paradigms** beyond MapReduce.
- Manages **compute, network, and memory resources** efficiently.

Why Use YARN?

- **Efficient dynamic allocation** of cluster resources, improving utilization.
- Enables parallel execution of **batch processing, streaming, and interactive queries**.
- Supports **various processing engines**, making Hadoop more flexible.

YARN Architecture Overview

- **Resource Manager:** Allocates resources across the cluster.
- **Node Manager:** Executes tasks on individual nodes.
- **Application Master:** Manages the lifecycle and resources for each application.
- **Container:** A resource package (RAM, CPU, Disk, Network) assigned to tasks



YARN Application Workflow

1. Client submits an application.
2. Resource Manager allocates a container for the Application Manager.
3. Application Manager registers with Resource Manager.
4. Application Manager requests containers from Resource Manager.
5. Node Manager launches containers.
6. Application executes inside the container.
7. Client monitors application status.
8. Application Manager unregisters from Resource Manager.

Architecture features of YARN :

YARN has become popular for the following reasons -

- With the scalability of the resource manager of the YARN architecture, Hadoop may manage thousands of nodes and clusters.
- YARN compatibility with Hadoop 1.0 is maintained by not affecting map-reduce applications.
- Dynamic utilization of clusters in Hadoop is facilitated by YARN, which gives better cluster utilization.
- Multi-tenancy enables an organization to gain the benefits of multiple engines at once.

Merits and Demerits of YARN

Merits:

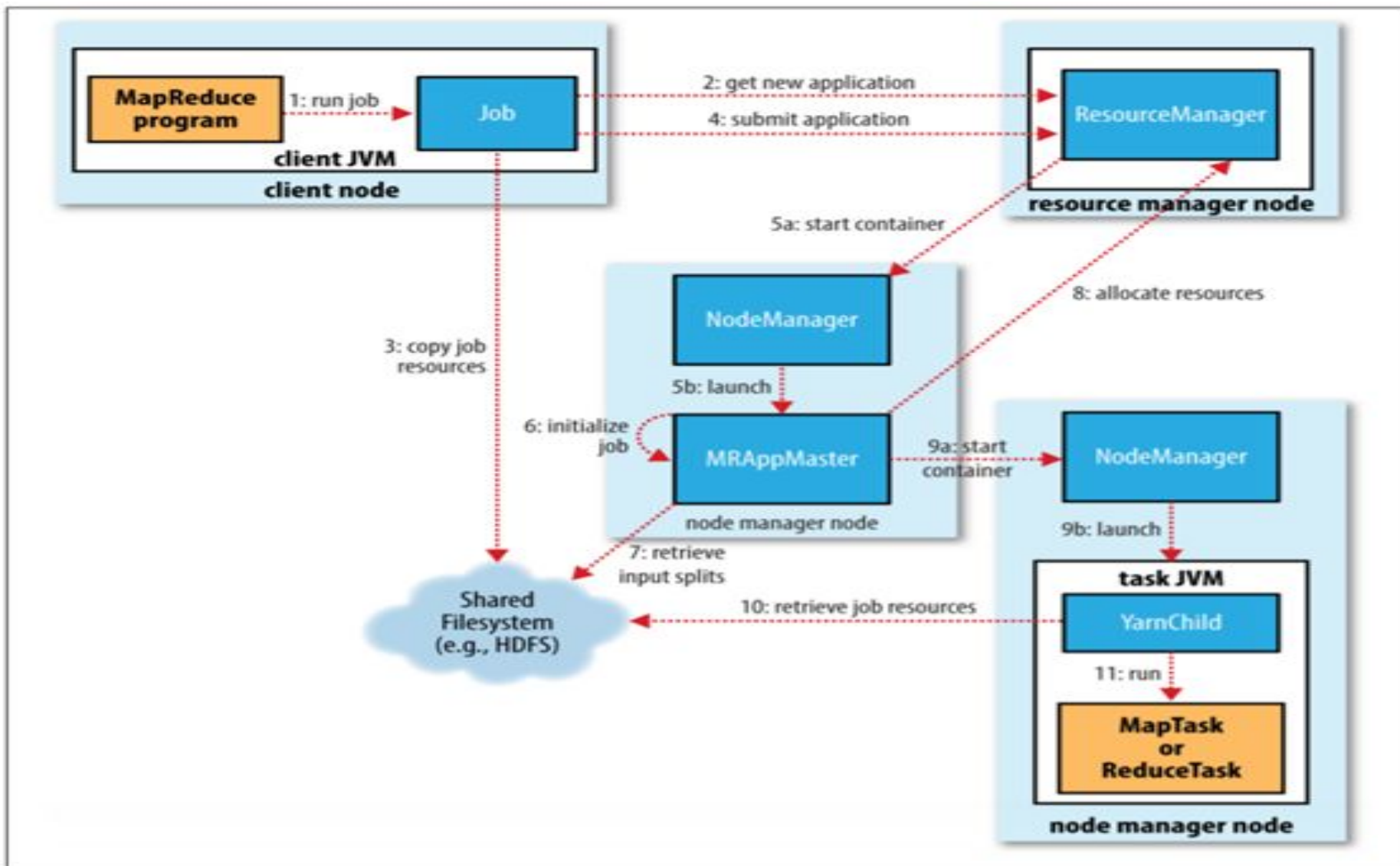
- **Scalability** : YARN is designed for large number of nodes.
- **Utilization** : Node manager manages a pool of resources, rather than a fixed number of the designated slots thus increasing the utilization.
- **Multitenancy** : Different version of MapReduce can run on YARN, which makes the process of upgrading MapReduce more manageable.

Demerit

- Availability - JobTracker is the only point of availability in Hadoop 1.0



YARN MAP REDUCE



YARN MAP REDUCE



Execution Process:

1. Run Job

A **MapReduce program** is submitted by the client, which initiates the job execution process.

2. Request New Application

The **Job** requests a new application from the **Resource Manager (RM)**.

3. Copy Job Resources

The client **copies the job resources** (JAR files, configuration files, and input data) to a **shared filesystem** (e.g., HDFS).

4. Submit Application

The **Job** submits the application to the **Resource Manager**.

5. Start Application Master (AM)

The **Resource Manager** requests a **Node Manager (NM)** to allocate a **container** to start the **MRAppMaster** (MapReduce Application Master).

The **Node Manager** launches the **Application Master** inside the container.

YARN MAP REDUCE



6. Initialize Job Execution

The **MRAppMaster** initializes the job execution process by interacting with HDFS to retrieve input splits.

7. Retrieve Input Splits

The **Application Master** retrieves input splits from HDFS to determine data distribution.

8. Allocate Resources for Tasks

The **Application Master** communicates with the **Resource Manager** to request necessary resources for task execution.

9. Launch Task Containers

The **Resource Manager** instructs the **Node Managers** to allocate and launch **containers** for executing **Map and Reduce tasks**.

10. Retrieve Job Resources

The task containers retrieve necessary job resources (JAR files, configurations, input splits, etc.) from the shared filesystem.

11. Execute MapTask or ReduceTask

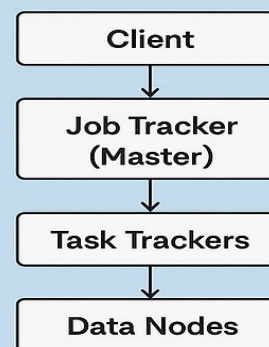
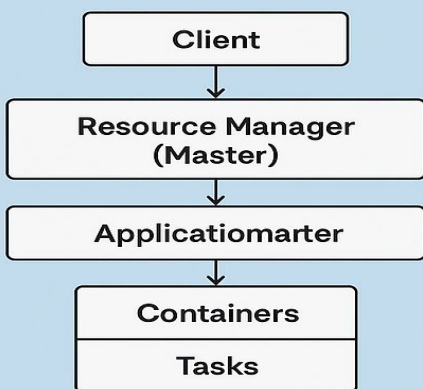
The **Task JVM** (YarnChild) executes either a **MapTask or ReduceTask** as part of the job.

Difference Between YARN and MapReduce

Feature	YARN	MapReduce
Hadoop Version	Introduced in Hadoop 2.x	Used in Hadoop 1.x
Architecture	Has Resource Manager, Node Manager, Application Master, and Containers	Uses Job Tracker and Task Tracker for processing
Scalability	More scalable , supports multiple applications simultaneously	Less scalable , only supports batch processing
Resource Allocation	Dynamic resource allocation based on application needs	Static resource allocation , which can lead to inefficient resource utilization
Processing Support	Supports batch processing, streaming, graph processing, and interactive queries	Supports only batch processing
Fault Tolerance	Better fault tolerance due to distributed resource management	Fault tolerance is less efficient
Flexibility	More flexible , can run various workloads (Spark, Tez, Flink, etc.)	Only supports MapReduce jobs
Resource Utilization	Optimized resource utilization , prevents bottlenecks	Resources can be underutilized due to fixed allocation

Difference Between YARN and MapReduce

Difference Between YARN and MapReduce



Resource Management	— Resource Manager and Node Managers
Processing Models	— Batch, streaming, graph, interactive
Scalability	— Highly scalable; multiple applications
Flexibility	— Multiple frameworks supported
	— MapReduce-only

Failures in Classic MapReduce



- **Three main types of failures:**

- Task Failure
- TaskTracker Failure
- JobTracker Failure

JobTracker is a single point of failure, requiring job restarts.

- **Task Failures:**

- **Child task failure:** Runtime exceptions in map/reduce tasks.
- **Streaming task failure:** Exits with a nonzero exit code.
- **Timeout:** No progress update leads to failure.
- **Solution:** Failed tasks are rescheduled.

- **TaskTracker Failures:**

- Stops sending heartbeats to JobTracker.
- Running tasks are lost.
- JobTracker reassigns incomplete tasks.
- TaskTracker may be blacklisted if failures exceed a threshold.

Failures in YARN



- **Three main failure types:**
 - Task Failure
 - Node Manager Failure
 - Resource Manager Failure
- **Improvements over MapReduce:**
 - Supports multiple applications with better fault tolerance.
- **Task Failures:**
 - Similar to MapReduce (runtime exceptions, timeouts).
 - Application Master detects failure and reschedules tasks.
- **Node Manager Failures:**
 - Stops sending heartbeats to Resource Manager.
 - Running tasks and Application Masters on that node are lost.
 - Resource Manager reassigns tasks to other nodes.

Resource Manager Failures in YARN

- **Most serious failure in YARN.**
- **Effects:**
 - No new applications or tasks can be launched.
 - Job progress tracking is lost.
- **Recovery Mechanism:**
 - Uses **checkpointing** to store state persistently.
 - Future versions improve fault tolerance.

Job Scheduling in Hadoop

Purpose of Hadoop Schedulers:

- Efficient resource utilization
- Performance enhancement

Challenges in Job Scheduling:

- **Energy Efficiency** – Large data centers consume high energy.
- **Load Balancing** – Unequal data partitioning leads to imbalance.
- **Mapping Scheme** – Efficient schemes reduce communication costs.
- **Automation & Configuration** – Incorrect settings impact performance.
- **Fairness** – Scheduling algorithms must allocate resources equitably.
- **Data Locality** – The closer the computation node, the faster the process.
- **Synchronization** – Managing intermediate data transfer efficiently.

FIFO Scheduler in Hadoop

Definition:

- **First In, First Out (FIFO)** is the **default scheduling policy** in Hadoop.
- Jobs are executed **in the order of submission, regardless of size or priority.**

How It Works:

- Applications are placed in a **queue**.
- The **first job** in the queue gets executed first.
- Next jobs wait until the previous job completes.



Advantages & Disadvantages of FIFO Scheduler



Advantages:

1. **Simple & easy to understand** – No complex configurations needed.
2. **Ensures job execution order** – Jobs are processed sequentially.

Disadvantages:

1. **Not suitable for shared clusters** – Can cause resource monopolization.
2. **Ignores job length & priority** – Large jobs delay smaller ones.
3. **Leads to resource imbalance** – Some jobs may starve for resources.

Fair Scheduler in Hadoop

- **Developed by Facebook** for **equal resource allocation**.
- If a **single job runs**, it gets **full cluster resources**.
- As more jobs arrive, resources are **fairly distributed**.

Uses Pools:

- Jobs are **grouped based on attributes** (e.g., username).
- Each pool has a **minimum share** of resources.
- **Unused resources** can be borrowed by other pools.

Prevents job overloading:

- Limits jobs if a single user submits too many.

Example

Company Scenario: DataCorp Inc.

DataCorp has three teams using the Hadoop cluster:

- **Analytics Team** – runs long data processing jobs at night.
- **Marketing Team** – runs quick ad-hoc reports during the day.
- **AI Team** – trains machine learning models which are resource-heavy.

Fair Scheduler: “Share it Fairly”

Let's say:

- All three teams are active at the same time.
- The cluster has 100 GB RAM and 100 vCores.

How it works:

- Each team gets ~33% of the resources.
- If the **Marketing Team** is idle (e.g., during the night), the **Analytics** and **AI Teams** automatically get **more resources**.
- When Marketing comes back online, the system **rebalances** to give everyone their fair share again.

Real-time benefit:

- No need for strict configuration.
- Resources are **efficiently utilized**, especially when some teams are inactive.

Best for: Dynamic environments with fluctuating workloads and many users.

Fair Scheduler - Pros & Cons

Advantages:

1. **Fair & dynamic resource allocation** across users.
2. **Faster response for smaller jobs** over large jobs.
3. **Limits concurrent jobs per user & pool** for better efficiency.

Disadvantages:

1. **Complex configurations** compared to FIFO.
2. **Ignores job weight**, leading to **unbalanced performance**.
3. **Limits on running jobs** under each pool.



Capacity Scheduler in Hadoop

- **Developed by Yahoo to maximize cluster utilization.**
- **Uses Queues:**
 - Each queue is assigned to an **organization**.
 - **Security mechanism** ensures users **only access assigned queues**.

Job execution:

- Guarantees **minimum capacity** for each queue.
- **Unused resources** are reallocated efficiently.
- Supports **priority-based job scheduling**.



Capacity Scheduler: “Reserved But Flexible”

Let's say the admin sets up queues like this:

- analytics-queue: 50% capacity
- marketing-queue: 20% capacity
- ai-queue: 30% capacity

How it works:

- During the day, **Marketing** gets at least 20%. Even if their jobs are small, that slice is **reserved**.
- If **Marketing** is not using its full 20%, **Analytics** or **AI** can **borrow** that unused capacity.
- However, if **Marketing** submits new jobs, they are **guaranteed** to get back their 20%.

Real-time benefit:

- Each department has **guaranteed access** to a minimum share.
 - Ideal for organizations with **SLA** (Service Level Agreements) or departmental quotas.
- Best for:** Organizations needing **strict control and guarantees** on resource usage.



Capacity Scheduler - Pros & Cons

Advantages:

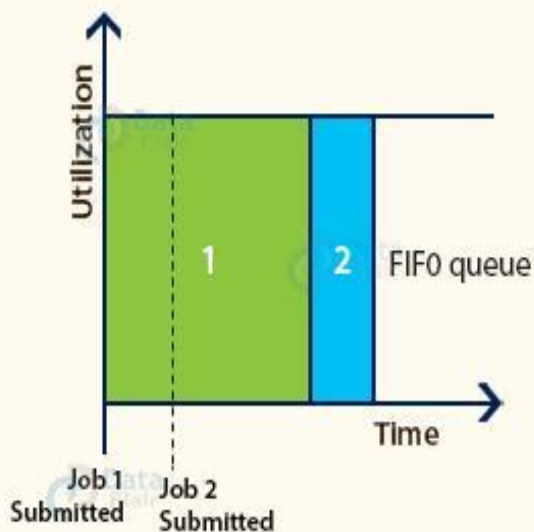
1. Maximizes resource utilization & throughput.
2. Reuses unused capacity dynamically.
3. Supports hierarchical queues & memory control.

Disadvantages:

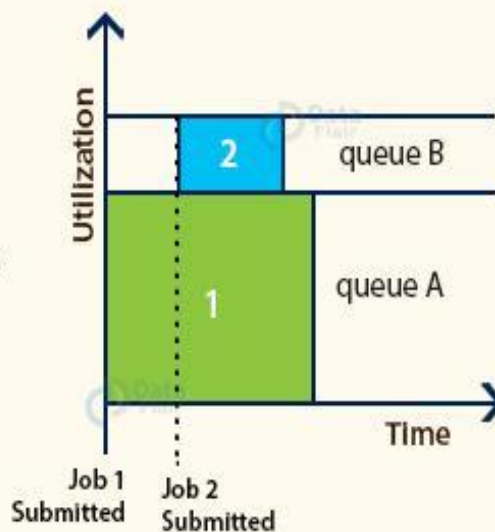
1. Most complex scheduler to configure.
2. Difficult to choose proper queues.
3. Stability issues with pending jobs.



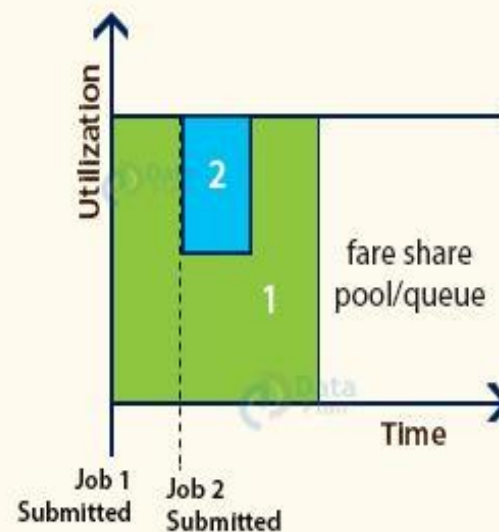
Hadoop Schedulers



(a) FIFO Scheduler



(b) Capacity Scheduler



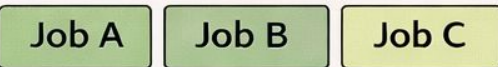
(c) Fair Scheduler

FIFO Scheduler

Job Queue
(Order of Arrival)



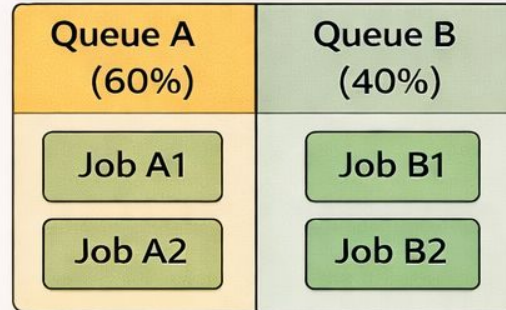
Execution:



→ One job at time (line system)

Capacity Scheduler

Cluster Resources (100%)



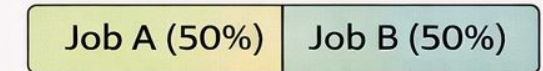
→ Fixed share for each queue

Fair Scheduler

Cluster Resources (Shared)



Step 1:



Step 3:



→ Equal sharing among jobs

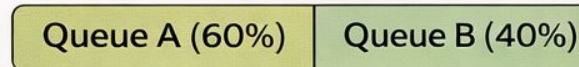
Job Queue
(Order of Arrival)



→ One job at time (line system)

FIFO Scheduler

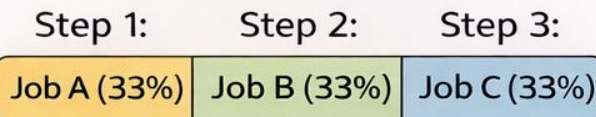
Capacity Scheduler



→ Fixed share for each queue

Capacity Scheduler

Fair Scheduler



→ Equal sharing among jobs

Fair Scheduler



Difference Between Fair Scheduler and Capacity Scheduler

Fair Scheduler (Facebook)	Capacity Scheduler (Yahoo)
Ensures fair resource sharing among users	Maximizes resource utilization and throughput
Distributes resources equally among jobs	Allocates resources based on predefined queues
Uses pools to group jobs based on user attributes	Uses queues , each assigned to an organization
Unused resources can be borrowed by other pools	Unused resources are reallocated dynamically
No strict priority mechanism; fair distribution	Supports priority-based job scheduling
Moderate complexity	Most complex scheduler
Best for shared clusters where fairness is needed	Best for organizations needing controlled resource allocation
Doesn't consider job weight , may cause imbalance	Difficult queue selection and stability issues with pending jobs

Shuffle and Sort in Hadoop



- **Shuffle:** Process of transferring sorted map outputs to reducers.
- **Sort:** Ensures that input to each reducer is sorted by key.

How it Works?

1. Map Phase:

1. Mappers process data and produce key-value pairs.
2. Output is stored in memory buffers and sorted by key.
3. If buffers overflow, data is spilled to local disk.

2. Shuffle Phase:

1. Once a Mapper finishes, sorted data is sent to the respective Reducer.
2. Ensures that all values for a given key reach the same Reducer.
3. Reduces network congestion by starting early data transfer.

3. Sort Phase:

1. Each Reducer receives shuffled data and sorts it again.
2. Sorted key-value pairs are processed for final output.
3. Output is written to **HDFS**.

Shuffle and Sort in Hadoop



Optimization: Using Combiner

- A **Combiner** is like a mini-reducer that **pre-aggregates** data before sending it over the network.
- Reduces **data transfer** and **improves performance**.
- Hadoop decides when and how often to use it.

Custom Shuffle & Sort Mechanism

- **Partitioner**: Controls data distribution among reducers.
- **RawComparator**: Handles sorting at Mapper and grouping at Reducer.

What is a Combiner?

- A mini-reducer that runs **after the Mapper**, before shuffling.
- Used for **local aggregation** to reduce data transfer.

Not guaranteed to run — **optional optimization**. **Combiner – Real Example**



Word Count Example

Input:

Hello world Hello Hadoop

Mapper Output (before Combiner):

<Hello, 1>, <world, 1>, <Hello, 1>, <Hadoop, 1>

With Combiner:

<Hello, 2>, <world, 1>, <Hadoop, 1>



Less data transferred → better performance

Shuffle & Sort Phase

Core Phase Between Mapper & Reducer

- Hadoop **shuffles** keys and **sorts** them before reducing.
- Two ways to customize:
 - **Partitioner**
 - **RawComparator**

Custom Partitioner

What is a Partitioner?

- Controls how keys are **distributed across reducers**.

Example: Grouping Sales by Region

<US, 200>, <US, 400>, <IN, 100>, <IN, 50>

Custom Logic:

if (key.equals("US")) return 0; else if (key.equals("IN")) return 1;

 All US data → Reducer 0, IN → Reducer 1.

RawComparator:

- Controls how keys are **sorted** and **grouped** in shuffle.
- Works at **byte-level** before deserialization.

Example: Sorting User Actions by Date

- Key: (userID, date)
- Comparator groups by userID and sorts by date

Reducer receives:

user1 → [actions sorted by date]



Task Execution in MapReduce

- The **MapReduce model** divides a job into smaller tasks that can run **in parallel**, significantly reducing total job execution time compared to sequential processing.

Speculative Execution in Hadoop

- **Definition:** Speculative execution is a mechanism to improve job performance by handling **slow-running tasks (stragglers)**.

How It Works:

- When a task is **executing slowly**, the **JobTracker (master node)** initiates another instance of that same task on a different node.
- The **first task to finish** is considered successful, and the **other is terminated**.

Purpose:

- Prevent a **single slow task** from delaying the entire job.
- Ensures that the job finishes **as quickly as possible**, even when some nodes underperform.

Key Points:

- **Only one copy** of a straggler can be speculated at a time.
- Speculative execution applies to **both map and reduce tasks**.
- It can be configured **globally (cluster-wide)** or on a **per-job basis**.

By **default**, speculative execution is **enabled** for both map and reduce tasks.

How Hadoop Identifies Stragglers:

- Hadoop assigns each task a **progress score** ranging from **0 to 1**.
- A task is marked as a straggler if:
 - Its progress is **less than the average**.
 - It has been running for at least **1 minute**.

Advantages of Speculative Execution in Hadoop

1. Improved Fault Tolerance

1. In large-scale clusters, **hardware failures, network issues, or overloaded nodes** are common.
2. Speculative execution helps **mitigate the risk** of job failure or delays caused by these issues by launching **redundant tasks**.

2. Faster Job Completion

1. A MapReduce job's total duration is often limited by the **slowest task** (a "straggler").
2. By launching a duplicate task, Hadoop ensures that **whichever finishes first is accepted**, thus **speeding up job completion**.

3. Better Resource Utilization

1. Instead of waiting for a slow node, Hadoop can use **idle resources** on other nodes to execute duplicate tasks.
2. This helps **maximize throughput** and keeps the cluster more **efficiently utilized**.

4. More Reliable Job Execution in Unstable Environments

1. In production environments with **thousands of nodes**, speculative execution provides a **safety net** against unpredictable slowdowns or failures.

Map and Reduce Functions in Hadoop

Function Signatures:

•Map Function:

$(K1, V1) \rightarrow \text{list}(K2, V2)$

- Takes an input key-value pair and produces a list of
- intermediate key-value pairs.

•Reduce Function:

$(K2, \text{list}(V2)) \rightarrow \text{list}(K3, V3)$

- Takes intermediate keys and list of values,
- processes them, and outputs final key-value pairs.

Type Rules:

- The **output types of the map function** (K2, V2) must match the
- **input types of the reduce function.**
- The **final output** of reduce (K3, V3) can have different types from
- both the map input and output

Input Formats in Hadoop

Hadoop supports various formats to ingest data from different sources like flat files or databases.

InputSplit and Record:

- **InputSplit**: A **logical chunk** of input data assigned to a map task.
- **Record**: A **key-value pair** derived from an InputSplit that the mapper processes.
- Splits and records are **logical** and not restricted to physical file structures.

InputSplit Properties:

- Defined by the InputSplit interface.
- Has:
 - **Length** in bytes.
 - **Storage locations** (e.g., hostnames where data resides).

Common InputFormat Classes

1. FileInputFormat (Base class):

- Used when input source is files.
- Responsible for:
 - Deciding which files to include.
 - Generating input splits.

2. TextInputFormat (Default InputFormat):

- Each record = one **line** of the file.
- **Key**: LongWritable → Byte offset of the line's start.
- **Value**: Text → Content of the line (no line breaks).

3. CombineFileInputFormat:

- Handles **many small files** efficiently by combining them into a **single split**.
- Reduces **overhead** of too many mappers when input files are small.

4. DBInputFormat:

- Used to read records from a **database**.

5. SequenceFileInputFormat:

- Used for **binary key-value pairs**.
- Sequence files are **splittable**, **compressible**, and efficient for binary data.
- **Variant**: SequenceFileAsBinaryInputFormat treats keys/values as **opaque binary**.



Splits vs. HDFS Blocks

- Input splits are **independent of HDFS block boundaries**.
- Example: A logical record (like a line of text) might **span multiple HDFS blocks**.
- Splits are **line-aware** (for text), so data isn't lost even if a split crosses a block boundary.
- This can lead to **remote reads**, but overhead is minimal.

In every lecture, Faculty may talk on any of the following:

- **DCS & GSFCU Portal:** Remind students to use DCS, talk about it's features and instruct all to visit GSFCU website
- **Discipline, Values and Ethics.**
- **GSFC University – My University:** Emphasize the importance of taking ownership of the University.
- **Respect & Courtesy:** Remind students to be respectful to peers, teachers, and staff. Encourage polite language and behavior.
- **Punctuality:** Emphasize the importance of arriving on time to class and completing assignments by deadlines.
- **Classroom Rules:** Reinforce basic classroom rules, such as keeping mobile phones on silent and maintaining a clean environment.
- **Safety:** Remind students about safety protocols, whether it's physical safety in the classroom or digital safety when using online resources.
- **Positive Attitude:** Encourage a positive mindset, cooperation and supporting one another.

GSFCU & Values

Respect & Courtesy

Font

Type: Verdana
Size: 20 / 18 / 16
Colour: Black



Dress
Appropriately



Respect
Personal Space



Be Mindful
of Your Words



Respect
Coworkers' Time



Keep Your
Workplace Clean



Socialise with
Your Peers



Use Your
Mobile Wisely



Take
Responsibility